

Kristálytiszta Elektronika 2

A Kristálytiszta Elektronika tananyag fejlesztését az Európai Bizottság támogatta az Erasmus+ program keretén belül „2020-1-HU01-KA201-078779” számú „Elektronika és programozás témájú innovatív tananyag fejlesztése a köznevelés számára” című projekt kapcsán.

Az Európai Unió
Erasmus+ programjának
társfinanszírozásával



A projektet az alábbi 4 intézményből álló nemzetközi partnerség valósította meg:

- Xtalin Mérnöki Tervező Kft. – Budapest
- ELTE Bolyai János Gyakorló Általános Iskola és Gimnázium – Szombathely
- Bolyai Farkas Elméleti Líceum – Târgu Mureș
- Pro Ratio Alapítvánnyal karöltve működő Madách Imre Gimnázium – Šamorín



XTALIN



Szerzői jogok:

Jelen tananyag az Xtalin Mérnöki Tervező Kft., mint koordinátor által vezetett partnerség szellemi terméke. Az anyagok oktatási felhasználásra készültek, ezért e célra szabadon felhasználhatóak; tartalmuk azonban nem módosítható, továbbfejleszthető az Xtalin Kft. írásban adott engedélye nélkül. Az anyagok újraközlése tartalmilag változatlan formában, kizárólag a tananyag szerzőinek és az eredeti tananyag elérhetőségének világos megjelölésével lehetséges, kizárólag az Xtalin Kft. írásban adott engedélyével.

Kapcsolat: <http://kristalytisztaelektronika2.hu/>
info@kristalytisztaelektronika.hu

14. Szenzorok kalibrálása

Írta: Palotás Gergely

Lektorálta: Horváth Tamás

Az előző tananyagrészen megismerhettünk különböző szenzortípusokat. Általánosságban elmondhatjuk, hogy ezek az érzékelők a környezet valamilyen állapotát vagy változását alakítják át villamos jellé, amit egy jelfeldolgozó egységnek továbbítanak. A fejlesztőpanelen ez a jelfeldolgozó egység a mikrovezérlő. A szenzor vagy a mérési módszer pontatlanságából, vagy a mérést befolyásoló egyéb tényezőkből adódóan a mért érték nem egyezik meg a valós értékkel, ezt mérési hibának nevezzük. A mérést akkor tekinthetjük teljesen precíznek, ha a hibaszámítást is elvégeztük hozzá. A mérési eredményben alapvetően két hibatípus jelenik meg. A rendszeres hiba a mérési módszer és a mérés kivitelezésének hibái. Ilyen például egy eszköz linearitási hibája, amikor az eszköz által szolgáltatott érték nem áll egyenes arányban a valós értékkel. A másik a véletlen hiba, aminek az oka vagy ismeretlen, vagy felderítése aránytalanul nagy ráfordítással járna, így ésszerűtlen a keresése. Erre egy jó példa a kvantálási hiba, amikor egy analóg jelet digitális jellé alakítunk és a jelszinteket egy adott tartományban azonos értékűnek veszünk. A véletlen hiba pontos értéke és előjele nem ismert. Ezzel ellentétben a rendszeres hiba nagysága és előjele meghatározható, így ezeket a kalibrálás segítségével csökkenteni tudjuk, ezáltal a mérésünk eredménye még közelebb kerül a mérendő mennyiség valós értékéhez.

Kitekintés:

A mérési hibákat eredetük, jellegük és formájuk szerint is csoportosíthatjuk. Az eredet szerint lehet például kivitelezési, vagy számítási hiba. A jellege szerint megkülönböztetjük a már említett rendszeres és véletlen hibákat. A formája alapján pedig beszélhetünk például megjelenítés szerinti hibákról, többek között ilyen az abszolút vagy relatív hiba. Az összes hibatípust most nem mutatjuk be, mert az egy külön tananyagrészt igényelne, de például a kvantálási hiba eredetét tekintve a mérés kivitelezéséből adódik, jellegét tekintve véletlen, és forma szerint abszolút hiba. Érdekes abba is egy kicsit belegondolni, hogy már azáltal, hogy valamilyen mérendő mennyiséget megmérünk, befolyásoljuk a mérendő mennyiséget. Például amikor egy folyadékba belemerítünk egy hőmérőt, akkor a hőmérő megváltoztatja az adott folyadék hőmérsékletét. Persze ha tudjuk, hogy ez milyen mértékben befolyásolja a végeredményt, akkor a mérésünk ezzel korrigálható.

KALIBRÁLÁS

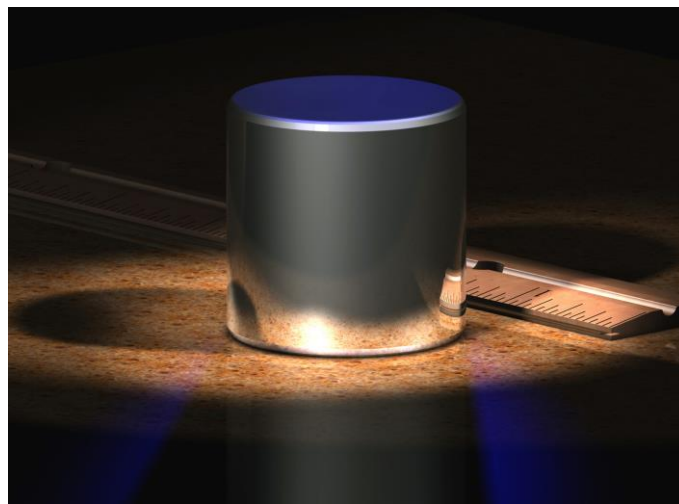
A hétköznapi életben sokszor találkozunk olyan helyzettel, amikor valamilyen eszközt kalibrálnunk kell. Nézzük meg pontosan mit is jelent a kalibrálás!

Definíció szerint a kalibráció az a művelet, amely során adott körülmények között megállapítjuk az összefüggést az érzékelő által mért érték és a mérendő mennyiség helyes értéke között. Ezt a helyes értéket egy kifejezetten más mérőeszközök ellenőrzésére használt eszközzel (etalon) biztosítjuk és célszerű a teljes mérési tartományon figyelembe venni. Amennyiben nem állnak rendelkezésre precíz

laboratóriumi eszközök, nem tudunk tökéletes kalibrációt végrehajtani, ettől függetlenül egyes hibafajtákat egyszerűbb módszerekkel is eliminálhatunk. A szenzorok esetében a szenzor által mért értékeket, a kalibráció során a program valós értékeknek felelteti meg, amely értékek a mért mennyiséget a kívánt formában és mértékegységben fejezik ki.

Kitekintés:

Az etalon francia eredetű szó, jelentése: hiteles, szabványos mérték. Ez egy mértékegység állandó, hiteles mintapéldánya. Az etalonok idővel változhatnak, hogy egyre pontosabban tudjuk meghatározni azokat. Például a tömeg SI-alapegységének, a kilogrammnak az etalonja is többször változott az idők során. Az első alkalommal, 1795-ben, úgy határozták meg, hogy egy kilogramm annyi víz tömegének felel meg, amennyi egy 0,1 méter élhosszúságú kockába fér a víz fagyáspontján. 1889-ben egy platina-irídium ötvözetből készült, 39 mm magasságú és átmérőjű henger tömegéként definiálták.



1. ábra: A kilogramm 1889-ben elfogadott etalonjának számítógépes képmása, ami egy 1 kg tömegűnek definiált platina-irídium henger.

Forrás: Gnash [CC BY-SA 3.0 (<https://creativecommons.org/licenses/by-sa/3.0/>)]
<https://commons.wikimedia.org/wiki/File:CGKilogram.jpg>

Nézzük meg a nullponti hiba (offset) kalibrálásának folyamatát egy digitális mérleg példáján keresztül. Ezek a mérlegek legtöbbször valamilyen nyúlásmérő bélyeg segítségével működnek. A bélyeg egy elektromos ellenállás, aminek az ellenállása a megnyúlásával arányosan változik. Tehát a definícióban szereplő mért érték ebben az esetben az ellenállás értéke, a mérendő mennyiség pedig a mérlegre helyezett test tömege, aminek hatására a bélyeg megnyúlik. Amikor bekapcsoljuk a mérleget, még nem teszünk rá semmit, ekkor az említett két értéket megfeleltetjük egymásnak, és utána tudunk bármit is mérni vele. Magyarul, a mérendő mennyiség helyes értéke nulla, és ezt hozzárendeli az ekkor mért ellenállás-értékhez. Persze ezzel csak a nullponti hibát küszöböljük ki, más hibatípusokat, mint például a linearitási hibát, nem. Vagyis, ha ezután ráteszünk 1 kilogramm tömeget, akkor még nem biztos, hogy a mérleg ennyit is fog mutatni. Az előbbieken egy különleges etalonról beszéltünk, hiszen készíthetnénk olyan mérleget is, amit egy pontosan 1 kilogramm tömegű test segítségével tudnánk kalibrálni. A legtöbb esetben ilyen etalon nem áll a rendelkezésünkre, így természetesen sokkal egyszerűbb, ha az etalon 0 kilogramm tömegű, tehát az a tény, hogy nem teszünk rá semmit. Igaz, még ekkor is van a mérleg tálcájának valamekkora tömege, de ez egy rendszeres hiba, ami könnyen meghatározható.

Rendben, de miért is kell elvégeznünk ezt a kalibrálást minden egyes alkalommal, amikor a mérleget bekapcsoljuk?

Itt játszik lényeges szerepet a definícióban szereplő "adott körülmények között" kifejezés. Például más hőmérsékleten más ellenállás értéket mérünk 0 kilogramm esetén, mert a nyúlásmérő bélyeg ellenállása változik a hőmérséklet függvényében. Vagy ha sokat használjuk a mérleget, akkor idővel a nyúlásmérő bélyeg elhasználódik és emiatt mérünk más értéket, mint új korában. Itt már látjuk is, hogy ezek például olyan mérést befolyásoló tényezők, amiket ki tudunk védeni. Ezért mindig az adott körülmények között kell elvégeznünk a kalibrálást.

HŐMÉRSÉKLETÉRZÉKELŐ-TÍPUSOK

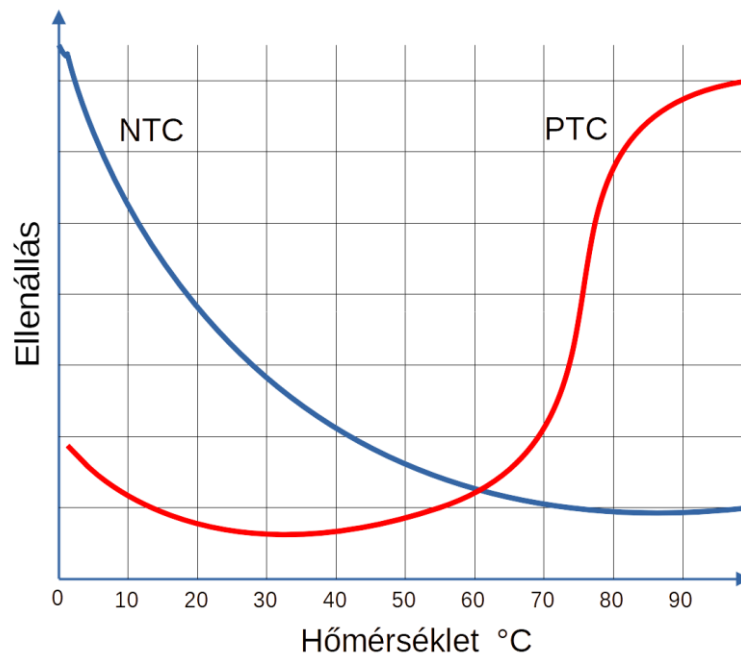
A kalibráció részletes folyamatát egy másik gyakorlati példán keresztül fogom bemutatni. Ehhez egy hőmérséklet méréséhez használható kapcsolást fogunk megépíteni. Először viszont nézzük meg, milyen hőmérsékletérzékelő-típust fogunk használni!

Fajtájukat és felhasználásukat tekintve több különböző hőmérsékletérzékelő közül választhatunk. A kapcsolásunkhoz mi az egyik legelterjedtebb típust, a termisztort fogjuk használni. A termisztor egy hőmérsékletfüggő ellenállás. Persze, hőmérséklet-változás hatására minden anyag változtatja az ellenállását valamilyen mértékben, viszont ezen eszközök esetében úgy választják meg az érzékelő anyagát, hogy az ellenállás változása szignifikáns legyen.

Az ellenállás-változás jellegét tekintve két fő termisztor fajtát különböztetünk meg. Ha a hőmérséklet növekedésével a termisztor ellenállása csökken, akkor negatív karakterisztikájú termisztorról beszélünk, ezt általában NTC (Negative Temperature Coefficient) vagy magyar terminológiában NTK (Negatív Termikus Koefficiens) rövidítéssel jelölik. Ha pedig a hőmérséklet növekedésével a termisztor ellenállása is növekszik, akkor pozitív karakterisztikájú termisztorról van szó, ennek rövidítése a PTC (Positive Temperature Coefficient) vagy PTK (Pozitív Termikus Koefficiens).

Kitekintés:

A PTC esetében az ellenállás növekedése általában nem igaz a teljes tartományra, azonban ez a jellege a meghatározó, illetve felhasználása során is ezt a tulajdonságát használjuk fel, ezért az előzőleg tett általánosítás.



2. ábra: NTC és PTC termisztor általánosított karakterisztikája

A mi kapcsolásunknál egy NTC termisztort fogunk használni. Fontos megjegyezni, hogy az NTC termisztor esetében ne lépjük túl a katalógusban engedélyezett maximális teljesítményt. Afölött ugyanis a keletkező hőmennyiséget már nem tudná megfelelően leadni, így a termisztor tovább melegedne, aminek a hatására az ellenállása még inkább lecsökkenne, és ez az öngerjesztő folyamat addig tartana, amíg az eszköz tönkre nem megy. Ezt az önerősítő folyamatot pozitív visszacsatolásnak hívjuk.

Kitekintés:

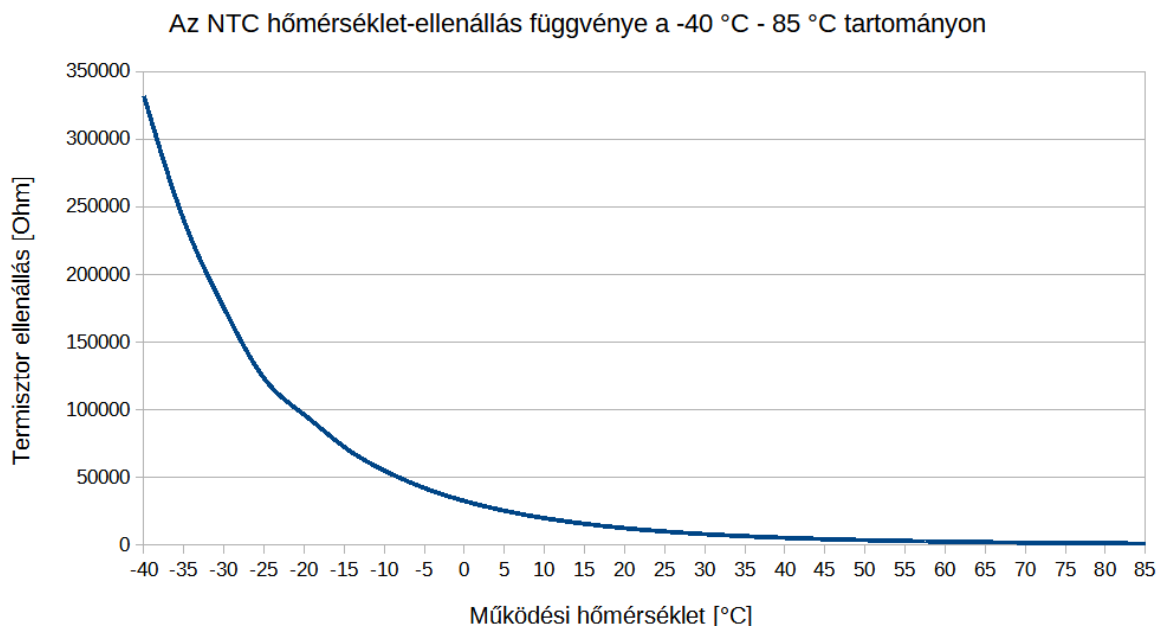
A termisztorok fajtájuktól függetlenül működés közben hőt adnak le, ezt hívjuk disszipált teljesítménynek. Minél nagyobb ez a teljesítmény, annál nagyobb a leadott hőmennyiség és természetesen ez annál nagyobb mértékben befolyásolja az általuk mért hőmérsékletet. Ez igaz a fejlesztőpanelen található I2C-s hőmérséklet-érzékelőre is. A villamos teljesítmény az alábbi összefüggés segítségével számítható ki:

$$P = U \cdot I = \frac{U^2}{R} = I^2 \cdot R$$

A fejlesztőpanel esetében a tápfeszültség adott, így a mérési elrendezésben használt ellenállás értékek megfelelő megválasztásával tudjuk biztosítani, hogy a termisztoron átfolyó áram ne érjen el egy kritikus értéket.

ANALÓG HŐMÉRŐ SZENZOR MŰKÖDÉSE

Az általunk választott NTC termisztor karakterisztikája az alábbi ábrán látható.



3. ábra: Az NTCLE400E3103H termisztor karakterisztikája

Ahogy már említettük, a hőmérséklet növekedésével a termisztor ellenállása csökken. Az ábráról az is szembetűnik, hogy a karakterisztika nem lineáris, tehát a hőmérséklet és az ellenállás változásának mértéke nem egyenesen arányos. Ez alapján úgy tűnik, hogy a hőmérséklet kiszámításához nem elegendő egy egyszerű lineáris egyenlet megoldása. Másrészt viszont lenne lehetőségünk a karakterisztika lineárisra tételére, akár további áramköri elemek hozzáadásával. Egyes beépített áramkörök például rendelkeznek ilyen megoldással, de a mi esetünkben az elrendezésben egy egyszerű feszültségosztót fogunk használni, ezt később majd részletesen megnézzük a mérési elrendezésnél. Emiatt meg kell küzdenünk a nemlinearitás által okozott nehézségekkel.

Kitekintés:

Manapság az érzékelők egyre nagyobb szerepet játszanak az életünkben, különösképp az automatizált rendszerekben. A legtöbb szenzor esetében a nemlinearitás egy régóta ismert probléma, ezért az érzékelő típusától és a nemlinearitás mértékétől függően különböző linearizációs módszerek léteznek. Ezek lehetnek valamilyen szoftveres megoldások, megvalósíthatóak további áramköri elemek beépítésével is, vagy akár ezeknek valamilyen kombinációjával. A szenzorok karakterisztikájának linearizációjával egyszerűsíthetjük a tervezést és a kalibrációt, illetve növelhetjük a mérés pontosságát.

A másik lehetőségünk tehát az, hogy megtaláljuk azt a nemlineáris egyenletet, ami leírja az adott szenzor karakterisztikáját. Egyes NTC szenzorok adatlapján megtalálható egy, az egész működési hőmérsékleti tartományt lefedő táblázat, amelyben a hőmérsékleti értékhez hozzá van rendelve az azokhoz tartozó ellenállás érték. A mi esetünkben az adatlap (3. oldal) tartalmazza ezen értékeket, de csak 5 °C-onkénti bontásban (-40 °C, -35 °C, ..., 0 °C, ..., 80 °C, 85 °C). Emiatt mindenképp szükségünk volna további számításokra, hogy az 5 °C-os tartományokon belüli hőmérsékletre milyen ellenállás értékek tartoznak. Persze ilyenkor már élhetnénk azzal a közelítéssel, hogy ezeken a tartományokon belül lineárisnak tekintjük a karakterisztikát, de most törekedjünk egy ennél pontosabb megoldásra.

Szerencsére a gyártó által megadott értékek segítségével ki tudjuk számolni a Steinhart-Hart egyenlet együtthatóit.

A Steinhart-Hart egyenlet a félvezetők ellenállásának változását modellezi különböző hőmérsékleteken.

$$\frac{1}{T} = A + B \ln R + C (\ln R)^3,$$

ahol

T a hőmérséklet (K),

R az ellenállás (Ω),

A, B, C a Steinhart-Hart együtthatók.

Az együtthatók kiszámításához három különböző hőmérséklet-ellenállás értékpárra van szükségünk. Érdemes ezekből kettőt a működési hőmérsékleti tartomány egyik, illetve másik végéhez közel, és egyet pedig a tartomány középső részéből választani, így a lehető legpontosabb eredményt kapjuk. Tehát összességében egy 3 ismeretlenes, lineáris egyenletrendszert kellene megoldanunk. Szerencsére ez egy elég gyakori probléma, így találunk kifejezetten erre a célra készített programokat, ahol bemeneti adatként csak meg kell adnunk a már említett három értékpárt, és eredményként megkapjuk az együtthatókat. Az egyszerűség kedvéért most megadjuk ezeket. Az általam választott három értékpár a következő:

$$T_1 = -40 [^{\circ}\text{C}], R_1 = 332\,094 [\Omega]$$

$$T_2 = 25 [^{\circ}\text{C}], R_2 = 10\,000 [\Omega]$$

$$T_3 = 85 [^{\circ}\text{C}], R_3 = 1070 [\Omega].$$

Az ezekből számított Steinhart-Hart együtthatók pedig a következők:

$$A = 1,135\,824\,991 \cdot 10^{-3}$$

$$B = 2,328\,905\,183 \cdot 10^{-4}$$

$$C = 0,936\,758\,577 \cdot 10^{-7}.$$

Az ellenőrzés kedvéért számoljuk ki a megismert egyenlet segítségével a 10 k Ω -hoz tartozó hőmérsékletet, és hasonlítsuk össze az adatlapban (3. oldal) megadott értékkel:

$$\frac{1}{T} = 0,001\,135\,8 + 0,000\,232\,89 \cdot \ln(10\,000) + 0,000\,000\,0936\,76 \cdot (\ln(10\,000))^3$$

$$\frac{1}{T} = 0,003\,354$$

$$T = 298,15 [K].$$

A végeredmény mértékegysége Kelvin. Ha a Celsius hőmérsékleti skálát a Kelvin segítségével definiáljuk, akkor 0 °C megfelel 273,15 Kelvin-nek. Tehát a mi eredményünk °C-ba átváltva: $T = 298,15 - 273,15 = 25$ °C. Pontosan azt kaptuk, amit vártunk.

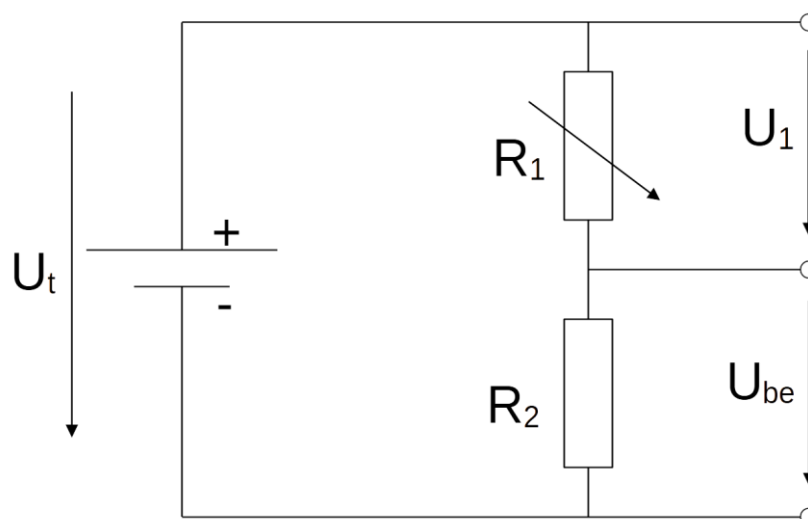
Valójában az egyenlet segítségével előre kiszámolhatnánk az összes ellenállás – hőmérséklet párt a kívánt felbontásban (például 0,1 °C) és minden mérésnél csak ki kellene keresnünk az adott tömbből. Ez jó megoldás lehet olyan esetben, ha a számítási kapacitás a szűk keresztmetszet, viszont van elég helyünk a Flash memóriában az értékpárok tárolására. Tegyük fel, hogy a mi esetünkben a tárhely a szűk keresztmetszet, nem szeretnénk felesleges helyet foglalni, ezért minden mérésnél az adott képlet segítségével fogjuk kiszámolni az aktuális hőmérsékletet.

MÉRÉSI ELRENDEZÉS

A mérés folyamata a következőképpen fog kinézni. A termisztort sorba kapcsoljuk egy referencia-ellenállással és ezekre tápfeszültséget kapcsolunk. A termisztor és az ellenállás között található (közös) pontot a mikrovezérlő analóg-digitális átalakítójának a bemenetére kötjük. Elvégezzük az átalakítást és a kapott feszültségértékből kiszámítjuk a termisztor aktuális ellenállását. A Steinhart-Hart egyenlet felhasználásával az ellenállás értékéből kiszámoljuk a hőmérsékletet. Végül a 9. tananyagrészen megismert OLED kijelző segítségével megjelenítjük a végeredményt. Az eddig leírtak persze még csak magát a hőmérséklet mérésének folyamatát írják le, ezt még természetesen ki kell egészítenünk a kalibráció folyamatával.

Nézzük meg akkor részletesen az imént leírtakat!

A feszültségosztó áramkör két egymással sorosan kapcsolt ellenállás, ahol a bemeneti feszültség megoszlik a két ellenálláson, vagyis az ellenállásokra eső feszültségek összege megegyezik a bemeneti feszültséggel.



4. ábra: Feszültségosztó áramkör

Az U_t a tápfeszültség, ami 3,3 V. Az R_2 egy 10 kΩ értékű ellenállás, az R_1 pedig maga a termisztor. Az U_{be} az a feszültség, amit a mikrovezérlő analóg-digitális átalakítójának bemenetére kapcsolunk, vagyis

amit mérni fogunk. A sorosan kapcsolt ellenállásokon átfolyó áram erőssége megegyezik, így ennek segítségével ki tudjuk számolni a termisztor ellenállását:

$$\frac{U_1}{R_1} = \frac{U_{be}}{R_2}$$

$$\frac{R_1}{U_1} = \frac{R_2}{U_{be}}$$

$$R_1 = \frac{U_1}{U_{be}} \cdot R_2$$

$$R_1 = \frac{U_t - U_{be}}{U_{be}} \cdot R_2$$

$$R_1 = \left(\frac{U_t}{U_{be}} - 1 \right) \cdot R_2$$

$$R_1 = \left(\frac{3,3 [V]}{U_{be}} - 1 \right) \cdot 10\,000 [\Omega].$$

Ebbe az egyenletbe már csak be kell helyettesíteni a mért bemeneti feszültség értékét, és megkapjuk a termisztor ellenállását. Ezt az ellenállást felhasználva pedig már csak a korábban említett Steinhart-Hart egyenletet kell megoldanunk.

A 11. tananyagrészen megismert analóg-digitális átalakítót 12 bites felbontásra konfiguráljuk be és 0 - 3,3 V-os feszültségtartományon fogunk mérni. Ez a tartomány a mi esetünkben természetesen egy elméleti tartomány, hiszen 3,3 V-ot csak akkor kaphatnánk, ha a termisztor ellenállása 0 Ω , 0 V-ot pedig csak akkor, ha az ellenállása végtelen nagy lenne. A termisztor az adatlap ajánlása alapján -40 °C és +85 °C közötti hőmérsékleti tartományban használható, a szélsőértékeken az NTC ellenállásának az értéke 332 094 Ω és 1 070 Ω . Az előző képletet átrendezve, és ezeket az értékeket behelyettesítve kiszámolható, hogy mi valójában a 0,0965 V és 2,981 V közötti feszültségtartományban fogunk bemeneti értékeket mérni, ami 3,3 V - (2,981 V - 0,0965 V) = 0,4155 V-tal kevesebb, mint a 3,3 V-os tartomány. Ez azt is jelenti, hogy nem használjuk ki teljesen az analóg-digitális átalakító kivezérlési tartományát, olyan mintha kisebb felbontásban használnánk azt, ami a pontosság csökkenését eredményezi.

A 12 bites felbontás alapján kiszámolhatjuk az LSB-t (Least Significant Bit, legkisebb helyiértékű bit), ami megmutatja, hogy mekkora az a legkisebb átlagos eltérés a bemeneten, ami a kimenet legkisebb helyiértékű bitjének a megváltoztatásához szükséges. Tehát az LSB értéke meghatározza a feszültségmérés pontosságát:

$$LSB = \frac{3,3 [V]}{2^{12}-1} = \frac{3,3 [V]}{4\,095} = 0,000\,81\,V = 0,81\,mV.$$

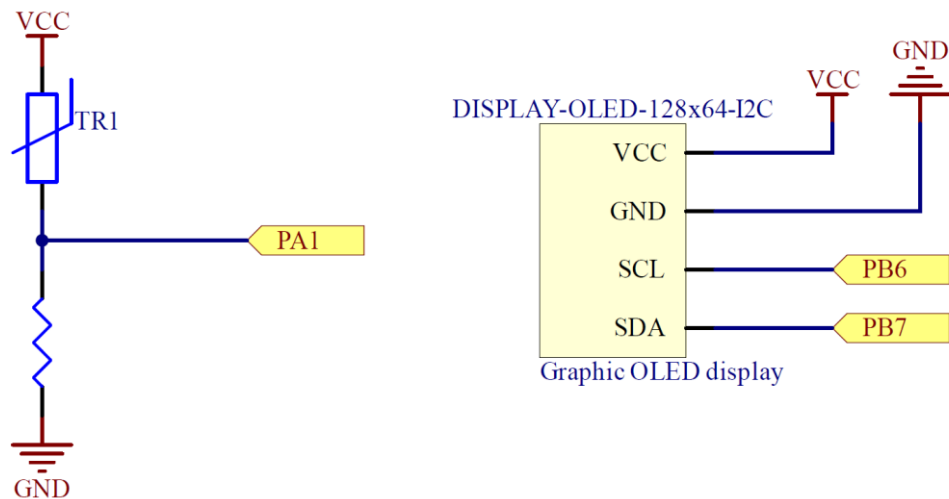
Az LSB-t és a korábban kiszámolt valós bemeneti feszültségtartományt felhasználva kiszámolható, hogy az elméleti 0 - 4095 helyett a 119 - 3680 tartományban fognak elhelyezkedni a konverziós értékeink.

Az, hogy a feszültség mérésének pontossága a hőmérséklet esetében milyen pontosságot jelent, attól függ, milyen hőmérséklet-tartományban mérünk. Minél kisebb a karakterisztika meredeksége, vagyis minél kisebb mértékben változik az ellenállás a hőmérséklet-változás hatására, annál nagyobb hibát fog nekünk jelenteni. A karakterisztikát megnézve ez azt jelenti, hogy minél nagyobb hőmérsékletet mérünk, annál kisebb lesz a mérés pontossága. A mi esetünkben szobahőmérséklet közelében fogjuk a mérést elvégezni, ezért nézzük meg, hogy ebben az esetben milyen hibát okoz az analóg-digitális átalakítás. Ezt úgy tehetjük meg, hogy a 25 °C-on mért bemeneti feszültség értékét megnöveljük egy LSB-vel, és kiszámoljuk az ahhoz az értékhez tartozó hőmérsékletet. Ahogy korábban már kiszámoltuk, ezen a hőmérsékleten a termisztor ellenállása 10 k Ω , pont amennyi a másik (R_2) ellenállás, így egyenlő mértékben oszlik el rajtuk a feszültség, tehát az átalakító bemenetére eső feszültség 1,65 V. Ha tehát ehelyett mi $1,65 + \text{LSB} = 1,65081$ V-ot mérünk, akkor az ellenállás 9 990,187 Ω . A Steinhart-Hart egyenletet megoldva 298,172 K-t, azaz 25,022 °C-t kapunk. Ebben az esetben tehát 0,022 °C az átalakításból eredő hiba. Összehasonlításképpen, 80 °C-nál ez a pontatlanság már 0,062 °C lenne. Összességében mondhatjuk azt, hogy a hőmérsékletmérést egy tizedesjegy pontossággal fogjuk végezni, és akkor az említett hibákat elhanyagolhatónak tekinthetjük.

KALIBRÁCIÓ FOLYAMATA

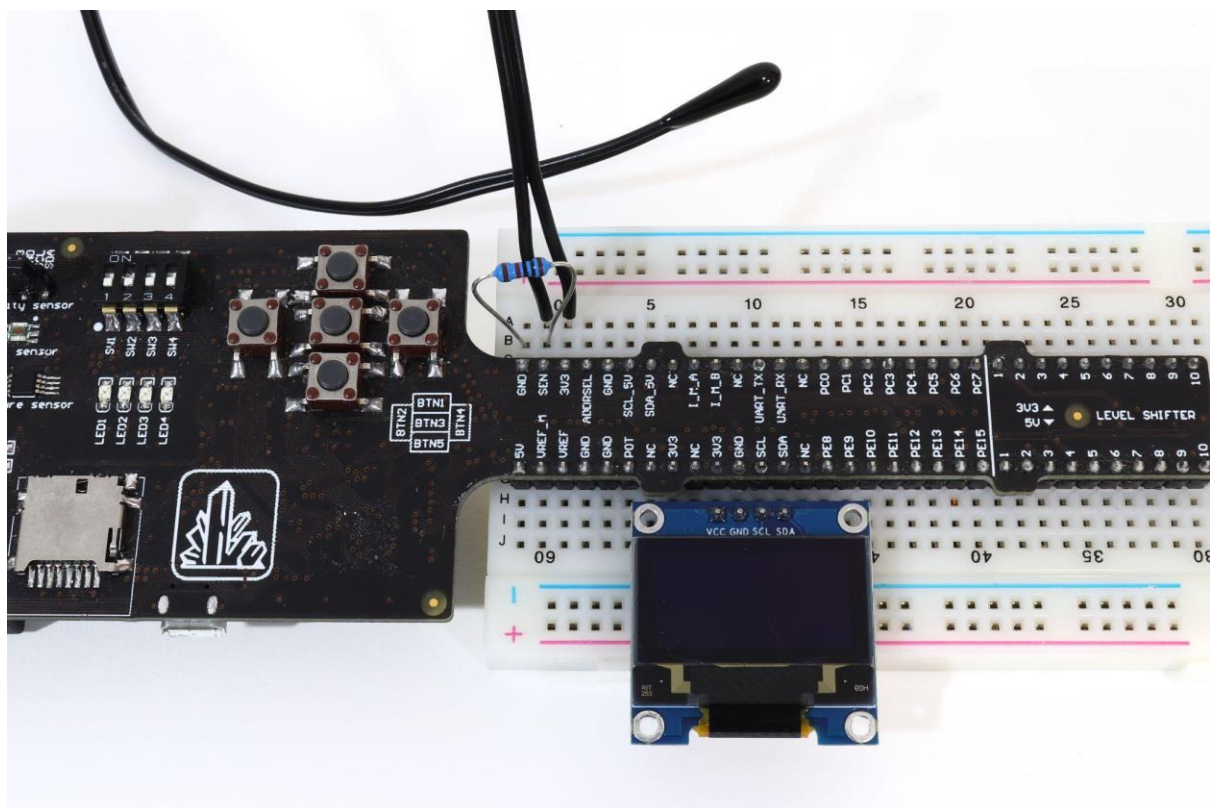
A kalibrálást 0 °C-os víz segítségével fogjuk elvégezni. Ezt akár otthon is könnyen elő tudjuk állítani úgy, hogy lefagyasztunk vizet, majd azt a fagyasztóból kivéve várunk, amíg elkezd megolvadni. Amikor már víz is van a jég mellett, akkor a jég-víz keverék 0 °C-os, hiszen a halmazállapot-változás 0 °C-on megy végbe. Az általunk választott termisztort kialakításából adódóan könnyen bele tudjuk meríteni a 0 °C-os vízbe. A termisztorunk válaszüzeje az adatlap (1. oldala) szerint kb. 7 másodperc, viszont ez egy speciális esetnek a válaszüzeje. A mi esetünkben ennél kicsit több idő szükséges, hogy a jó értéket mérjük, de a kijelzőn is látni fogjuk, amikor elérjük a 0 °C körüli hőmérsékletet, és ha a mért érték már stabil, akkor indíthatjuk a kalibrációt. Ekkor megállapítjuk a mért érték és a mérendő mennyiség helyes értéke közötti összefüggést, ami ebben az esetben az általunk számított hőmérséklet és a helyes érték, vagyis a 0 °C közötti különbséget jelenti. Ezt a különbséget eltároljuk, és a kalibráció befejezését követő mérések eredményéből levonjuk mindaddig, amíg új kalibrációt nem hajtunk végre.

Építsük meg a következő kapcsolást:



5. ábra: A termisztor és az OLED kijelző bekötésének kapcsolási rajza

Ha mindent jól csináltunk, valahogy így kell kinéznie:



6. ábra: Az elkészült mérési elrendezés

A következő lépésben készítjük el a programot. A korábbi tananyagrészek és az eddig leírtak alapján valójában megismertünk mindent, ami a program elkészítéséhez szükséges. Most az általunk készített példakód segítségével mutatjuk be a fontosabb programrészeket, de nyugodtan próbáljátok meg önállóan elkészíteni az egyes kódrészeket, vagy akár az egész programot.

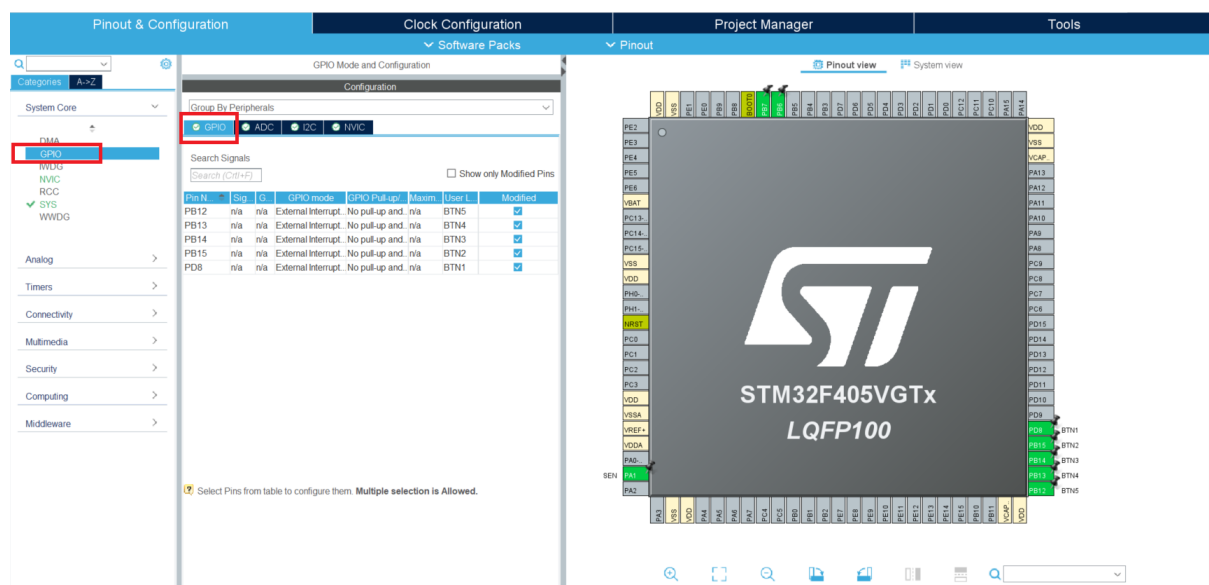
A példakódban a kalibrálás indítását, illetve az OLED kijelzőn történő megjelenítés vezérlését a nyomógombok segítségével végezzük el. A programban két képernyő van definiálva, a kezdőképernyő (SCRN_Start) és az analóg-digitális átalakításhoz tartozó információkat megjelenítő képernyő

(SCRN_ADC). Utóbbihoz három különböző megjelenítési mód tartozik. Az első az aktuális hőmérsékletet mutatja (DM_Temp), a második a kalibráció során eltárolt korrekciós értéket (DM_Diff), a harmadik pedig a konverziós eredményt (DM_ConvRes).

```
// lehetséges képernyő üzemmódok
enum Screen
{
    SCRN_Start = 0,
    SCRN_ADC = 1,
};

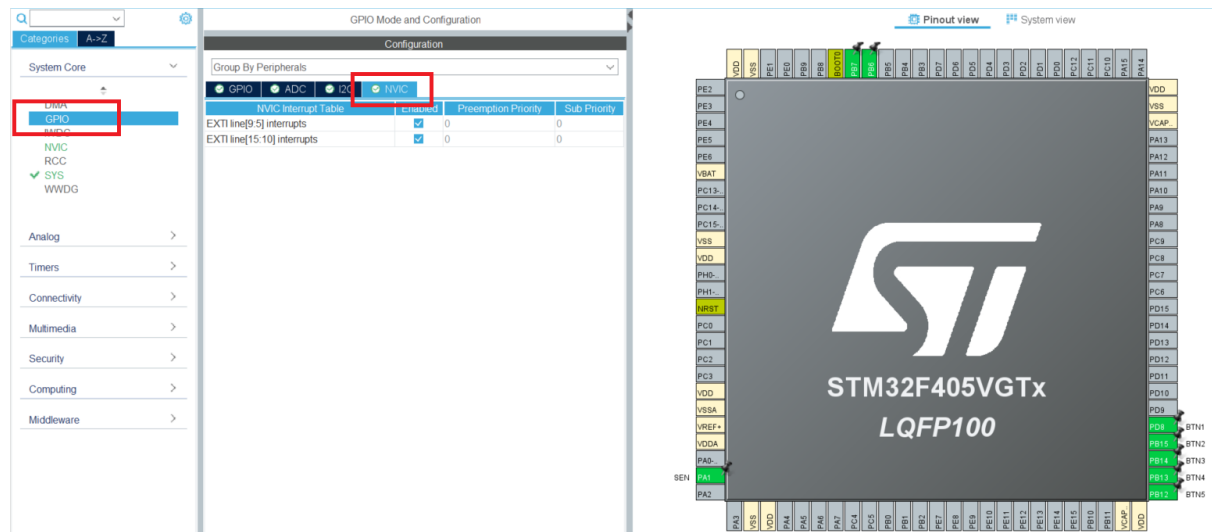
//a SCRN_ADC képernyő megjelenítési módjai
enum ADC_DisplayMode
{
    DM_Temp = 0,
    DM_Diff = 1,
    DM_ConvRes = 2
};
```

Először inicializáljuk a nyomógombokhoz tartozó lábakat: BTN1 (PD8), BTN2 (PB15), BTN3 (PB14), BTN4 (PB13), BTN5 (PB12). A nyomógombok pergesmentesítését megszakítás és időzítő segítségével valósítjuk meg, ezért a portokat nem egyszerű bemenetre, hanem külső megszakításos módra állítjuk a CubeMX-ben, ami az alábbi ábrán is látható:



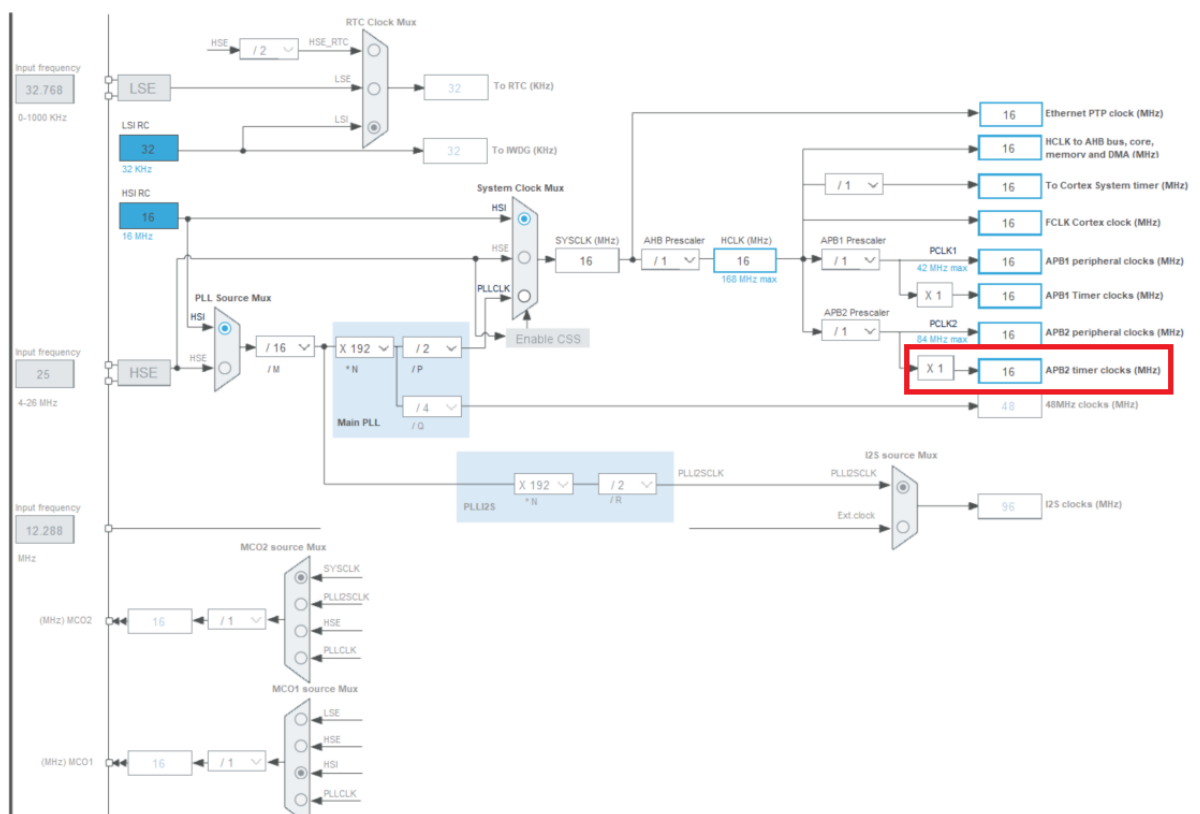
7. ábra: CubeMX láb beállítások - GPIO fül

A pergésmentesítés röviden úgy néz ki, hogy a gomb megnyomásakor megszakítás történik. A megszakításkezelő függvényben elindítunk egy időzítőt, és a "lejárata" kezelő függvényben megvizsgáljuk, hogy a gomb még mindig nyomva van-e. Ha igen, akkor tényleges gombnyomás történt és befejeződött a pergés. Ahhoz, hogy a megszakításokat használni tudjuk, azokat is engedélyezni kell.



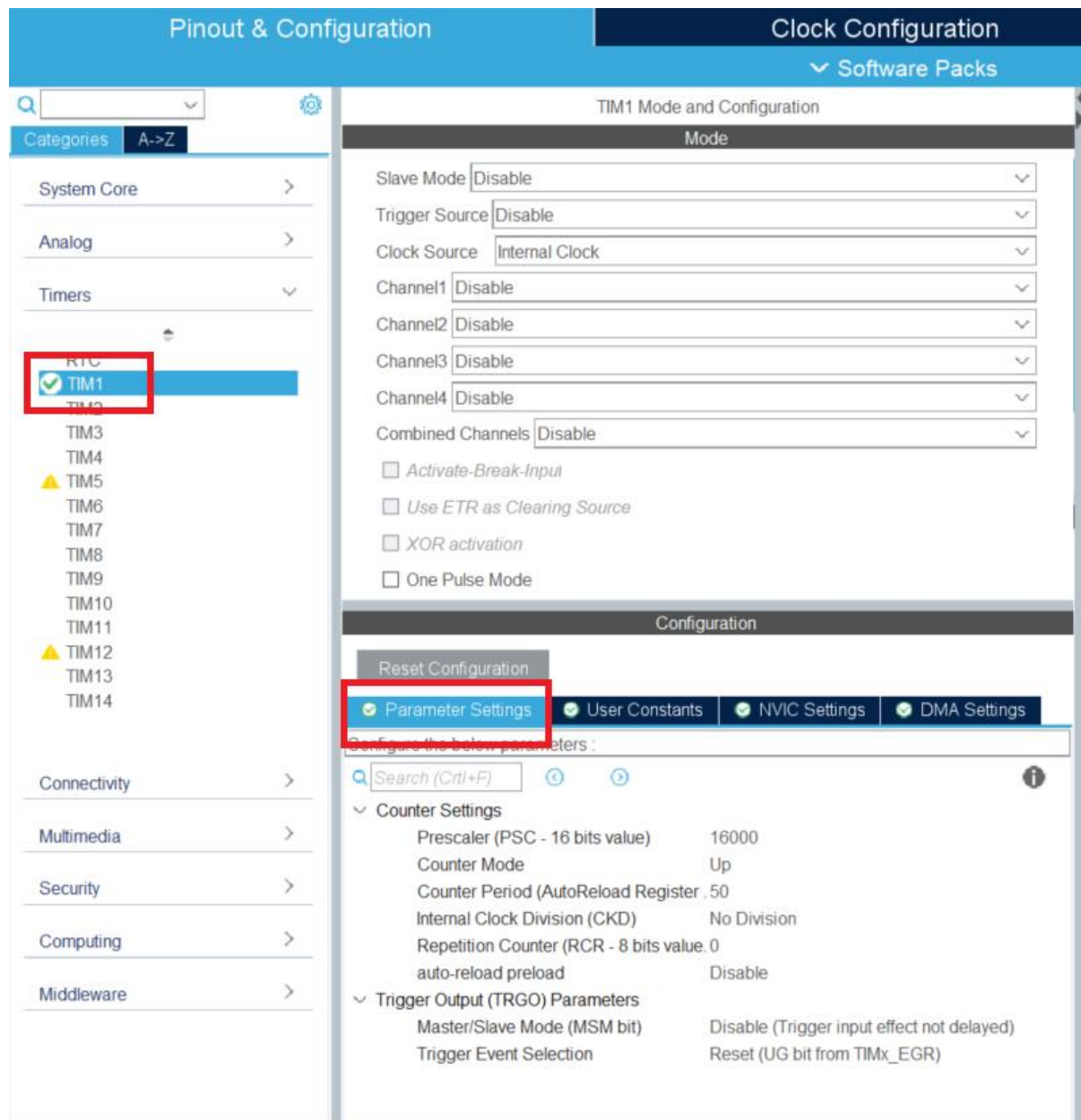
8. ábra: CubeMX láb beállítások - NVIC fül

Az időzítő inicializálásánál fontos, hogy a periódusidőt jól válasszuk meg, hiszen csak azt a gombnyomást tekintjük érvényesnek, amelynél az időzítő lejártakor a gomb még nyomva van. Egy normál gombnyomásnak körülbelül 50 ms felel meg. Az időzítőnk órajelét könnyen kideríthetjük. A mikrovezérlő adatlapjának 19. oldalán látjuk, hogy az általunk használt időzítő (TIM1) a mikrovezérlő ABP2 buszán helyezkedik el. A CubeMX órajel beállítások (clock configuration) fülén pedig látjuk, hogy ezen a buszon 16 MHz az időzítő órajele.



9. ábra: CubeMX órajel beállítások (Clock Configuration)

Az előosztó (prescaler) értéke 16 000, tehát 1 kHz-es frekvenciával 50 ms alatt éri el az 50-es értéket.



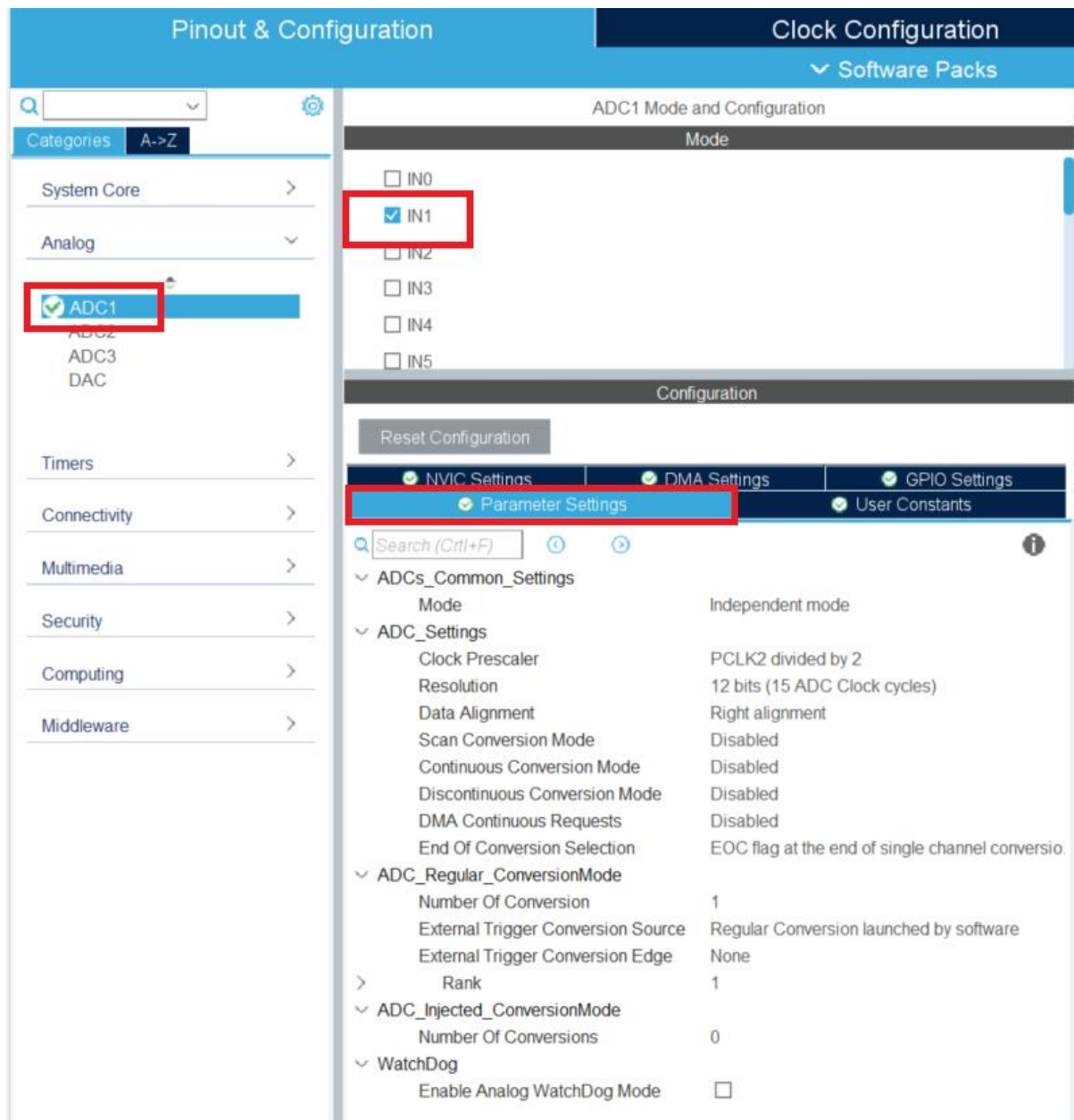
10. ábra: CubeMX időzítő beállítások

Az időzítő „lejártát” kezelő függvénybe definiáljuk a nyomógombok működését. Például a középső gomb (BTN3) segítségével fogjuk tudni végrehajtani a kalibrálást, ha a „SCRN_ADC” képernyőn „DM_Temp” módban vagyunk, illetve ezzel tudjuk törölni az eltárolt kalibrációs értéket „DM_Diff” módban.

```
//művelet indítása
else if (HAL_GPIO_ReadPin(GPIOB, BTN3_Pin) == GPIO_PIN_SET)
{
    //Temp képernyőnél a kalibrálás indítása
    if (current_screen == SCRN_ADC && display_mode == DM_Temp)
```

```
{  
    //a mérendő érték és a mért érték különbségének tárolása  
    temperature_correction = 0.0 - temperature_calculated;  
}  
//Diff képernyőnél a kalibrálás törlése  
else if (current_screen == SCRN_ADC && display_mode == DM_Diff)  
{  
    temperature_correction = 0.0;  
}  
}
```

A 11. tananyagrészen megismertek alapján inicializáljuk az analóg-digitális átalakítót 12 bites felbontásra. Ahogy a kapcsolási rajzon is látható, az A-D átalakító bemenetéhez a SEN (PA1) lábat használjuk.



11. ábra: CubeMX A-D átalakító beállítások

A 9. tananyagrészen tanult felhasználásával inicializáljuk az OLED kijelzőt. A kapcsolási rajzról leolvashatjuk, hogy a kijelzőhöz az I2C1_SCL (PB6) és I2C_SDA (PB7) lábakon található I2C kommunikációs interfészt fogjuk használni.

A különböző információk kijelzéséhez használt függvények segítségével egy szöveget tudunk majd kiírni az első sorba, illetve egy szöveget vagy numerikus értéket a második sorba.

```
//szöveg megjelenítése
```

```
void DisplayText(char* text_first_line, char* text_second_line, int unit)
```

```
{
```

```
    //előző kijelző törlése
```

```
OLED_Fill(Black);

//szöveg kiírása az első sorba
OLED_SetCursor(0, 0);
OLED_WriteString(text_first_line, Font_11x18, White);

//szöveg kiírása a második sorba
OLED_SetCursor(0, 36);
OLED_WriteString(text_second_line, Font_11x18, White);

//mértékegység kiírása
switch (unit)
{
case Celsius:
    OLED_DrawCircle(SSD1306.CurrentX + 4, 38, 2, White);
    OLED_SetCursor(SSD1306.CurrentX + 8, 36);
    OLED_WriteString("C", Font_11x18, White);
    break;
case NoUnit:
    break;
default:
    break;
}

//megjelenítés
OLED_UpdateScreen();
}

//szöveg és hozzá tartozó integer érték megjelenítése
void DisplayTextAndInteger(char* text, const int value, int unit)
{
    //integer érték átalakítása szöveggé
    char temp_string[64] = "";
    int ret = snprintf(temp_string, sizeof(temp_string), "%d", value);
}
```

```

    if ((ret < 0) || (ret >= sizeof(temp_string)))
    {
        strcpy(temp_string, "Conv Err");
    }

    DisplayText(text, temp_string, unit);
}

//szöveg és hozzá tartozó float érték megjelenítése
void DisplayTextAndFloat(char* text, const float value, int unit)
{
    //float érték átalakítása szöveggé egy tizedesjegy pontossággal
    char temp_string[64] = "";
    int ret = snprintf(temp_string, sizeof(temp_string), "%.1f", value);
    if ((ret < 0) || (ret >= sizeof(temp_string)))
    {
        strcpy(temp_string, "Conv Err");
    }

    DisplayText(text, temp_string, unit);
}

```

A program while(1) ciklusában, attól függően, hogy melyik képernyő van kiválasztva, a megfelelő műveleteket hajtjuk végre és az ahhoz tartozó képernyőt jelenítjük meg. Ez azért hasznos, mert ha új funkcionalitást szeretnénk létrehozni a programunkban, akkor készíthetünk egy új képernyőt, tehát könnyű a programunkat kibővíteni. A képernyők között a fel (BTN1) és le (BTN5) nyomógombokkal tudunk váltani. Az ADC képernyő esetében fél másodpercenként A-D átalakítást hajtunk végre. Ha az eredményünk ingadozik, elvégezhetünk több átalakítást is mérési ciklusonként, és az eredményeket átlagolhatjuk egy egyszerű számtani középérték kiszámításával. A példakódból származó alábbi kódrészletben például egy 16-szoros átlagolást használunk. Ha ezt ki akarjuk kapcsolni, állítsuk a "SAMPLE_NUMBER" értéket 1-re. Gyakorlatilag ekkor is működni fog az átlagolás, de csak egyetlen érték átlagát vesszük.

```

// az A-D átalakítás elvégzése és az aktuális képernyő megjelenítése
void ConvertAndDisplay()
{

```

```
// változó a mért adc értékek összegzésére
float cumulated_result = 0.0;

// az átlagoláshoz használt átalakítások száma
const unsigned int SAMPLE_NUMBER = 16;
// kommunikáció ellenőrzésére változó létrehozása
HAL_StatusTypeDef error_code = HAL_OK;
for (unsigned int i = 0; i < SAMPLE_NUMBER; ++i)
{
    // analóg-digitális átalakítás indítása
    HAL_ADC_Start(&hadc1);
    error_code = HAL_ADC_PollForConversion(&hadc1, 100);
    if (error_code != HAL_OK)
    {
        DisplayTextAndInteger("Error:", error_code, NoUnit);
        HAL_Delay(1000);
        break;
    }
    cumulated_result += HAL_ADC_GetValue(&hadc1);
}
// kommunikáció sikerességének ellenőrzése
if (error_code != HAL_OK)
{
    return;
}
// átlagolt eredmény kiszámítása
adc_result = cumulated_result/SAMPLE_NUMBER;
// hőmérséklet kiszámítása a kapott adc értékből
temperature_calculated = CalculateTemperature(adc_result);
// kalibrálási korrekció figyelembevétele
float temperature = temperature_calculated + temperature_correction;

// az eredmény megjelenítése a kijelző módjától függően
if (display_mode == DM_Temp)
```

```
{  
    DisplayTextAndFloat("Temp:", temperature, Celsius);  
}  
else if (display_mode == DM_Diff)  
{  
    DisplayTextAndFloat("Diff:", temperature_correction, Celsius);  
}  
else if (display_mode == DM_ConvRes)  
{  
    DisplayTextAndInteger("ADC:", adc_result, NoUnit);  
}  
}
```

Az A-D átalakítás eredményét felhasználva kiszámoljuk a hőmérsékletet.

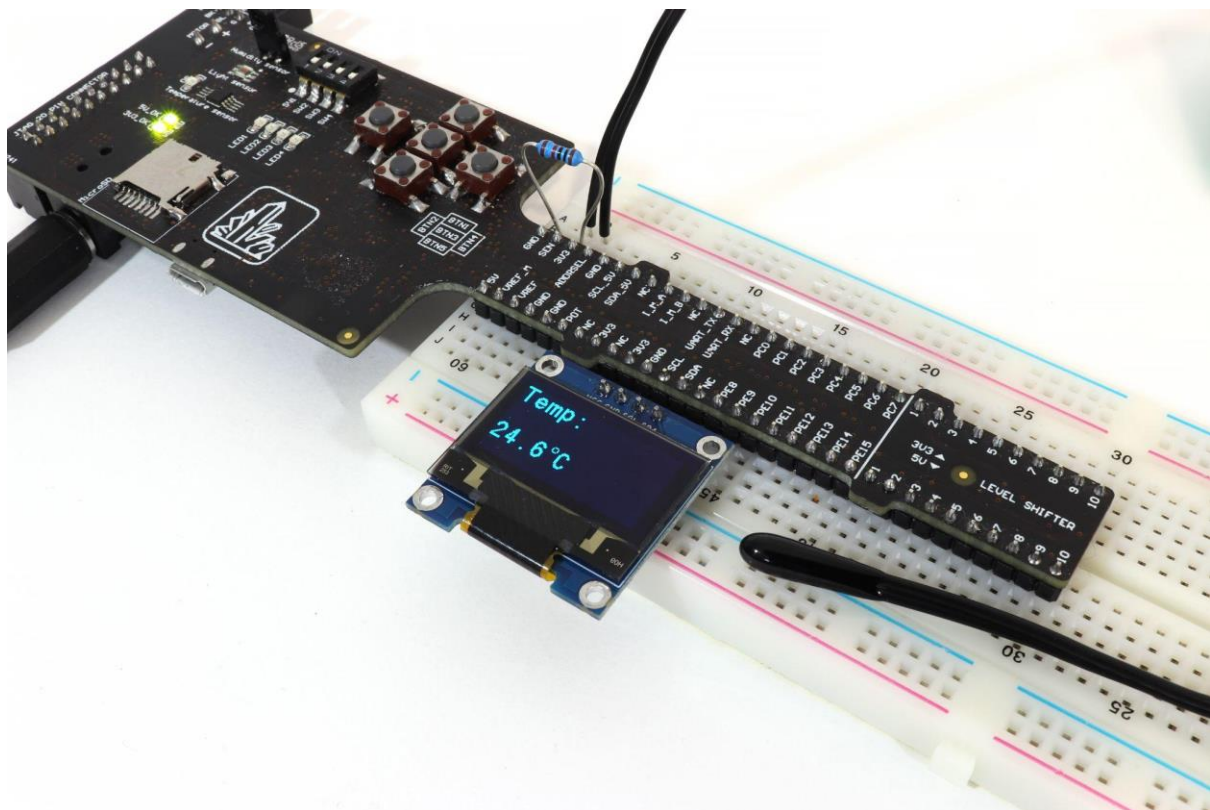
```
//a hőmérséklet számítása a konverziós eredményből  
float CalculateTemperature(uint32_t conversion_result)  
{  
    //nulla fok Kelvin-skála szerint  
    const float ZERO_IN_KELVIN = 273.15;  
  
    //Steinhart-Hart együtthatók  
    const float A = 1.135824991E-3f;  
    const float B = 2.328905183E-4f;  
    const float C = 0.9367585770E-7f;  
  
    //a termisztor ellenállásának kiszámítása  
    float thermistor_impedance = CalculateImpedance(conversion_result);  
  
    //hőmérséklet kiszámítása Steinhart-Hart egyenlet segítségével  
    float reciprocal_temp = A + B*log(thermistor_impedance) + C*  
    pow(log(thermistor_impedance),3);  
    float temp = pow(reciprocal_temp, -1);  
}
```

```
return (temp-ZERO_IN_KELVIN);  
}
```

Az ADC kijelző módjától függően megjeleníthetjük a hőmérsékletet, a kalibrációs értéket, illetve magát az átalakítás eredményét is. A kijelző módok között a bal (BTN2) és jobb (BTN4) nyomógombok segítségével válthatunk.

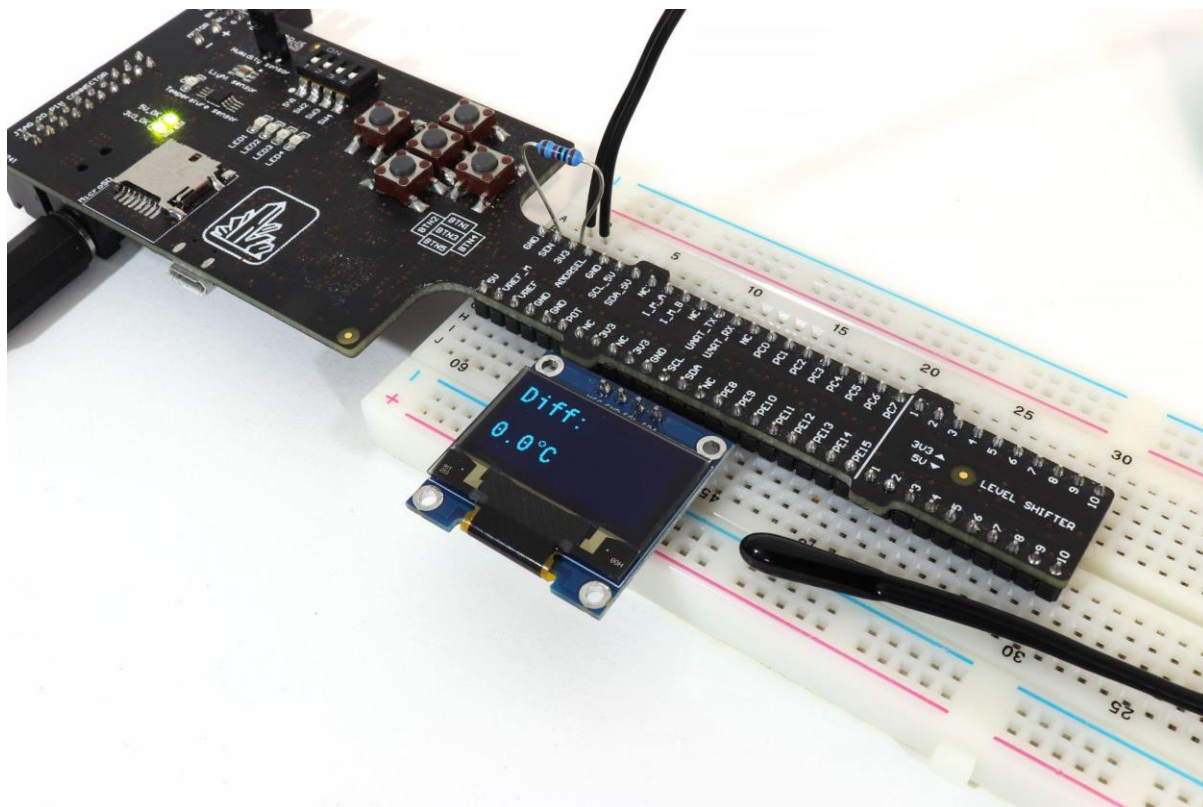
MÉRÉS

Most, hogy elkészültünk a programunkkal, végre kipróbálhatjuk, amiért eddig dolgoztunk, a kalibrációt. Indítsuk el a programot! Ha a példakód szerint dolgoztunk, akkor a kezdőképernyőt kell látnunk, ahonnan a fel (BTN1) vagy le (BTN5) nyomógommbal tudunk átnavigálni az ADC képernyőre. Ekkor egy „Temp” feliratot fogunk látni, és az aktuális hőmérsékletet.



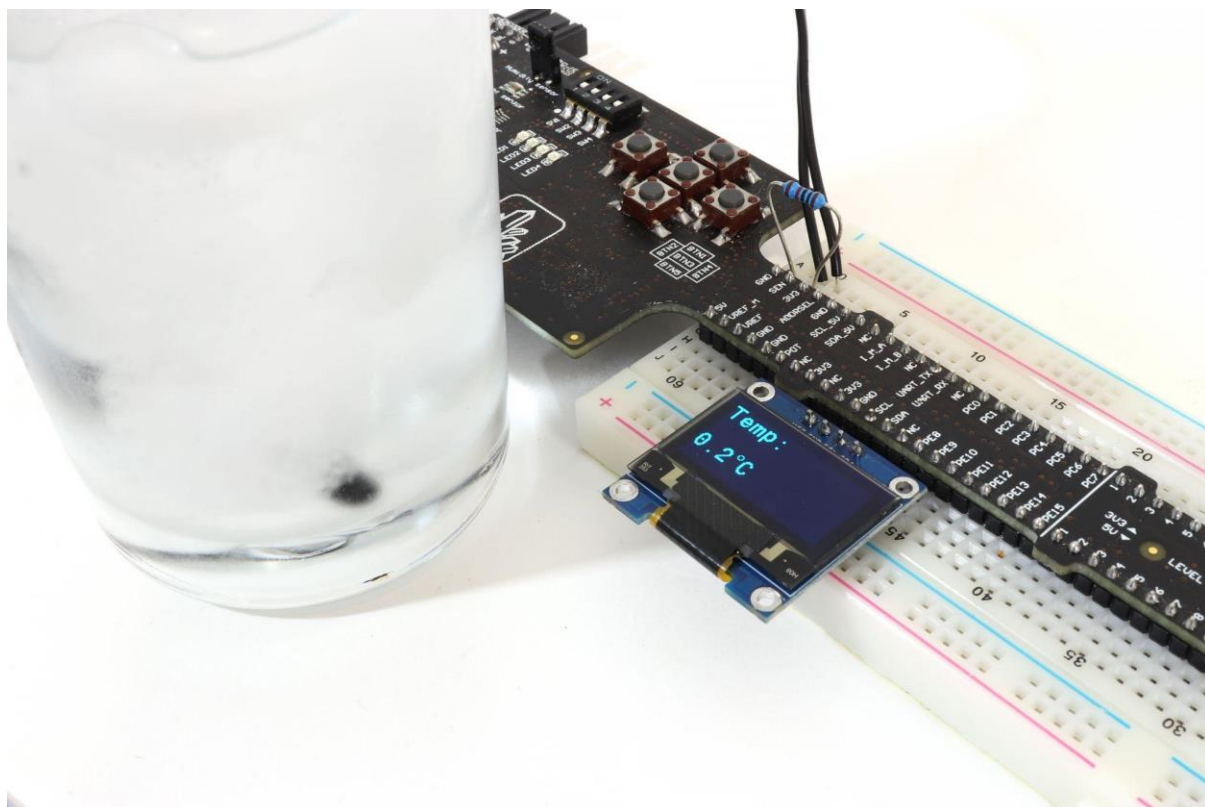
12. ábra: Az aktuális hőmérséklet a kalibráció előtt

Ha most a jobb (BTN4) nyomógommbal átváltunk „Diff” módba, akkor láthatjuk, hogy a kalibrációs érték 0, hiszen még nem kalibráltunk.



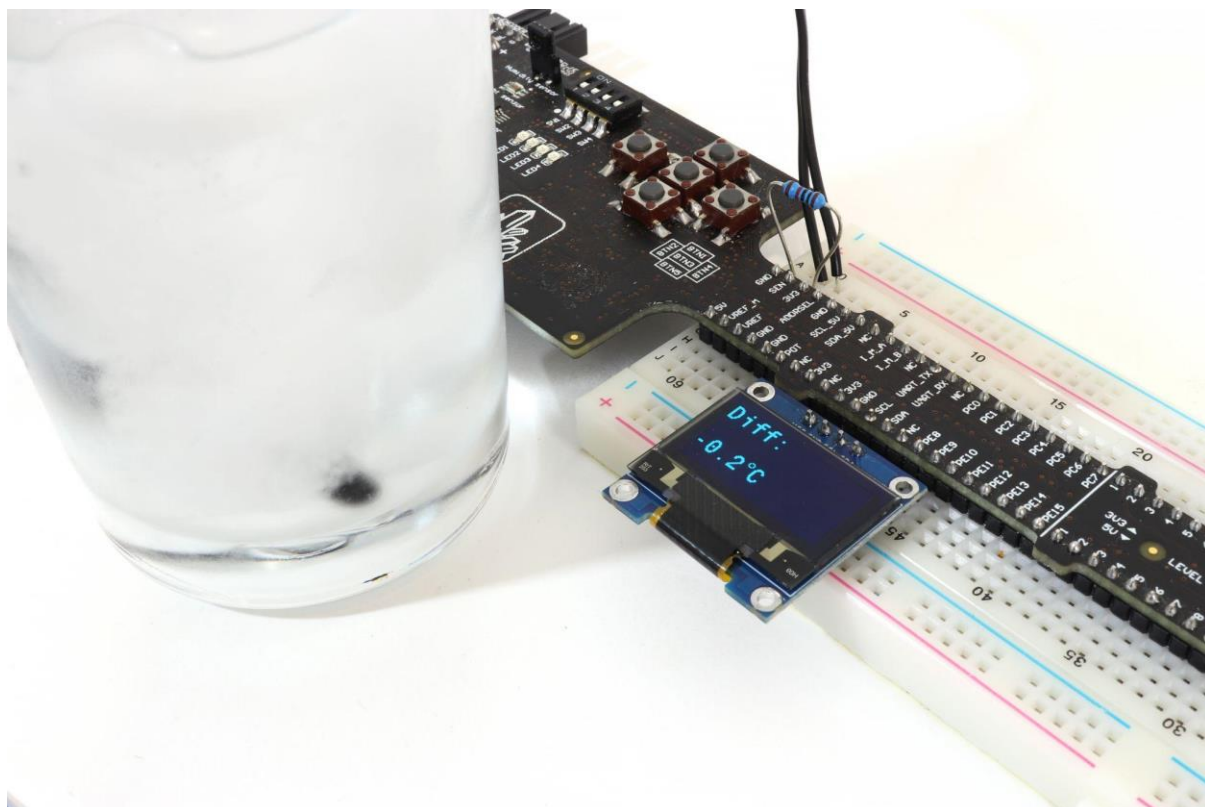
13. ábra: A kalibrációs érték a kalibráció előtt

Térjünk vissza „Temp” módba, hogy lássuk az aktuális hőmérsékletet, és kezdjük el a kalibrációt a 0 °C-os víz segítségével, amit a korábban leírtaknak megfelelően előkészítettünk. A termisztort merítsük bele a jég-víz keverékbe.



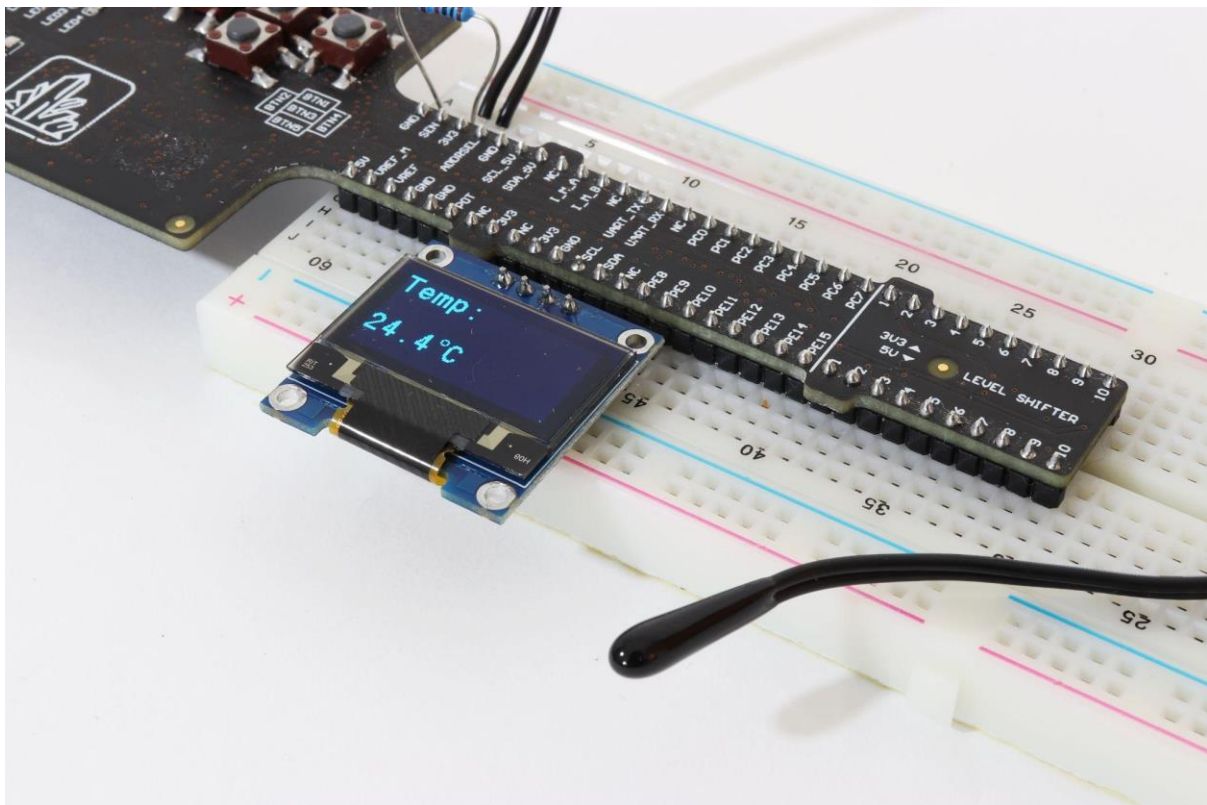
14. ábra: Kalibráció 0 °C-os víz segítségével

Várjunk, amíg az aktuális hőmérsékletünk eléri a 0 °C körüli, stabil hőmérsékletet, és ekkor a középső (BTN3) nyomógombot használva végezzük el a kalibrációt, vagyis tároljuk el azt az értéket, amivel a következő mérések eredményét a program korrigálni fogja. „Diff” módban meg is nézhetjük, mennyi ez az érték.



15. ábra: A kalibrációs érték a kalibráció után

A mi esetünkben tehát $0,2\text{ }^{\circ}\text{C}$ volt a különbség a mért érték és a mérendő mennyiség helyes értéke között. Tehát a kalibrációt elvégezve a program $0,2\text{ }^{\circ}\text{C}$ -ot fog levonni a további mérések eredményéből. Ha a termisztort kivesszük a jég-víz keverékből, akkor láthatjuk a pontos aktuális hőmérsékletet.



16. ábra: Az aktuális hőmérséklet a kalibráció után

ÖSSZEFOGLALÁS

Ebben a tananyagrészen megismerkedtünk a kalibráció fogalmával, illetve megnéztük egy gyakorlati példán keresztül magát a folyamatot. Érdekes megjegyezni, hogy szigorúan a definíciót tekintve a kalibráció nem jelenti a mérési hibák kiküszöbölését, "csak" egy összefüggést állapítunk meg azon értékek között, amit aktuálisan, adott körülmények között mérünk, illetve amit éppen mérünk kellene. Azt, hogy a további méréseket ezen összefüggés alapján hogyan korrigáljuk, már nem tartalmazza. Persze a gyakorlatban ez a két dolog sokszor összefonódik. A kalibrációt felhasználva olyan mérési hibákat is kiküszöbölhetünk, amelyeknek az okát egyébként viszonylag könnyen megszüntethetnénk. Például a mérési elrendezésünkönél mi egy 10 k Ω -os referencia ellenállással számoltunk. De tudjuk, hogy az ellenállások értéke egy adott tűrésen belül változhat. Az általam használt ellenállás tűrése 1%, és megmérve 9 950 Ω -os értéket kaptam. 0 °C esetén ez a különbség körülbelül 0,08 °C-os eltérést eredményez. Persze most könnyedén át lehetett volna írni a programban a referencia ellenállás értékét, de gondoljunk arra, hogy ha ebből az eszközből legyártanánk több százezer darabot, és ugyanazt a programot töltenénk rájuk, akkor már nem tudnánk minden egyes eszköz esetében a pontos ellenállás-értéket használni.

Tehát a kalibrációra azért van szükségünk, hogy gondoskodjunk a mérés minőségéről, és biztosítsuk, hogy az adott eszköz megfelelően működik.